САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
Факультет программной инженерии и компьютерной техники (ПИиКТ)
Дисциплина: «Расширенный курс по бэкенд-разработке в среде Node.JS»

ОТЧЕТ ПО ДОМАШНЕЙ РАБОТЕ №2
«Проектирование и технический дизайн API»

Выполнил:
студент гр. К3340 (поток БР1.1)
Галилей Кирилл Дмитриевич

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

1. ВАРИАНТ

Вариант 1. «Приложение для бронирования столиков в ресторанах»

Ключевая функциональность системы согласно заданию:
- Вход (аутентификация с выпуском JWT-токенов доступа)
- Регистрация (создание аккаунтов клиентов и менеджеров с хэшированием паролей)
- Личный кабинет пользователя (профиль, контактные данные, история)
- Поиск ресторанов с фильтрацией по кухне, расположению и цене
- Страница с информацией о ресторане (меню, фото, отзывы, столики)
- История бронирований пользователя (создание, просмотр, отмена)

2. ЗАДАЧА

1) Опишите все функциональные и нефункциональные требования к вашему бэкенд-приложению
2) Выберите формат реализации API, которого вы будете придерживаться: REST API или Backend For Frontend
3) Спроектируйте все эндпоинты вашего API в формате OpenAPI-схемы с учётом реализованной диаграммы БД
4) Опишите тело каждого запроса и тело каждого ответа, продумайте возможные ошибки, возвращаемые из API

3. ХОД РАБОТЫ

3.1. Анализ требований к бэкенд-приложению

Функциональные требования:
- Регистрация и аутентификация с выдачей JWT токена
- Управление личным профилем пользователя
- Поиск ресторанов с фильтрацией (кухня, локация, цена, рейтинг)
- Управление столами и меню заведения менеджером
- Бронирование столиков с проверкой слотов и защитой от овербукинга
- Публикация отзывов и автоматический пересчет рейтинга ресторана

Нефункциональные требования:
- Время отклика 95% запросов < 150 мс
- Stateless JWT авторизация в заголовке Authorization: Bearer <token>
- Валидация входных данных DTO и стандартизированный формат ошибок (RFC 7807)

3.2. Обоснование формата REST API

Выбран классический стиль REST API:
- Стандартизированная семантика HTTP-методов (GET, POST, PUT, PATCH, DELETE)
- Понятные статус-коды (200, 201, 204, 400, 401, 403, 404, 409, 500)
- Унифицированный контракт, подходящий для Web и Mobile клиентов без дублирования

3.3. Таблица спроектированных эндпоинтов (19 методов)

1.	POST	/api/v1/auth/register	(Public)	-> 201, 400, 409
2.	POST	/api/v1/auth/login	(Public)	-> 200, 400, 401
3.	GET	/api/v1/users/profile	(CLIENT, MANAGER)	-> 200, 401
4.	PATCH	/api/v1/users/profile	(CLIENT, MANAGER)	-> 200, 400, 401
5.	GET	/api/v1/restaurants	(Public)	-> 200, 400

6. POST   /api/v1/restaurants   (MANAGER)   -> 201, 400, 401, 403
7. GET   /api/v1/restaurants/:id   (Public)   -> 200, 404
8. PUT   /api/v1/restaurants/:id   (MANAGER)   -> 200, 400, 401, 403, 4
04
9. GET   /api/v1/restaurants/:id/tables   (Public)   -> 200, 404
10. POST   /api/v1/restaurants/:id/tables   (MANAGER)   -> 201, 400, 401, 403, 4
09
11. GET   /api/v1/restaurants/:id/menu   (Public)   -> 200, 404
12. POST   /api/v1/restaurants/:id/menu   (MANAGER)   -> 201, 400, 401, 403
13. DELETE /api/v1/menu-items/:id   (MANAGER)   -> 204, 401, 403, 404
14. POST   /api/v1/reservations   (CLIENT)   -> 201, 400, 401, 404, 4
09
15. GET   /api/v1/reservations/my   (CLIENT)   -> 200, 401
16. GET   /api/v1/reservations/restaurant/:id (MANAGER)   -> 200, 401, 403, 404
17. PATCH /api/v1/reservations/:id/cancel   (CLIENT, MANAGER) -> 200, 401, 403, 404, 4
09
18. GET   /api/v1/restaurants/:id/reviews   (Public)   -> 200, 404
19. POST   /api/v1/restaurants/:id/reviews   (CLIENT)   -> 201, 400, 401, 404, 4
09

3.4. Спецификация OpenAPI 3.0.3
Спецификация сохранена в файле openapi.yaml и полностью валидирована.

==========
4. ВЫВОД
==========
В ходе выполнения домашней работы №2 сформирован комплексный технический дизайн REST API д
ля сервиса бронирования столиков. Разработана и завалидирована спецификация OpenAPI 3.0.3,
 содержащая 19 эндпоинтов, покрывающих все функциональные требования Т3. Описаны DTO запро
сов и ответов, коды ошибок и механизмы авторизации по стандарту JWT Bearer.